拟合整理：

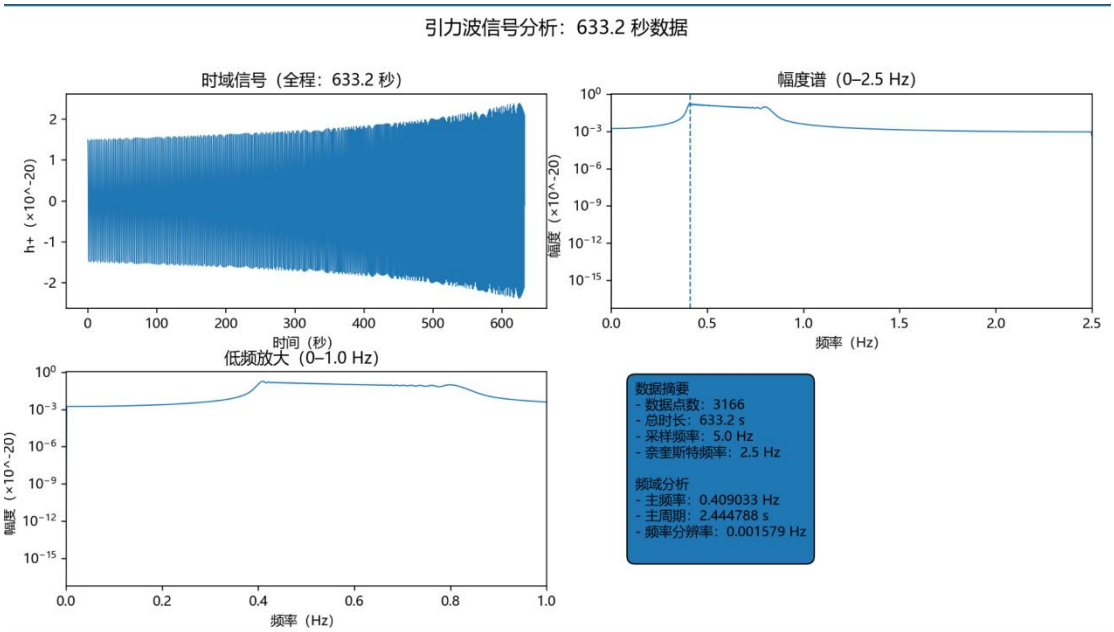
1. $dt = 0.2\ s$

共有 N=3166 个点，总时长为 T=633.2 s，采样频率为 $fs = 5.0$Hz

奈奎斯特频率为 fN=2.5 Hz

频率分辨率为 $\Delta f = 0.001579\ Hz$

对 h+(t)做去均值处理 $h_0(t) = h(t) - \langle h \rangle$，以避免直流分量影响低频谱。



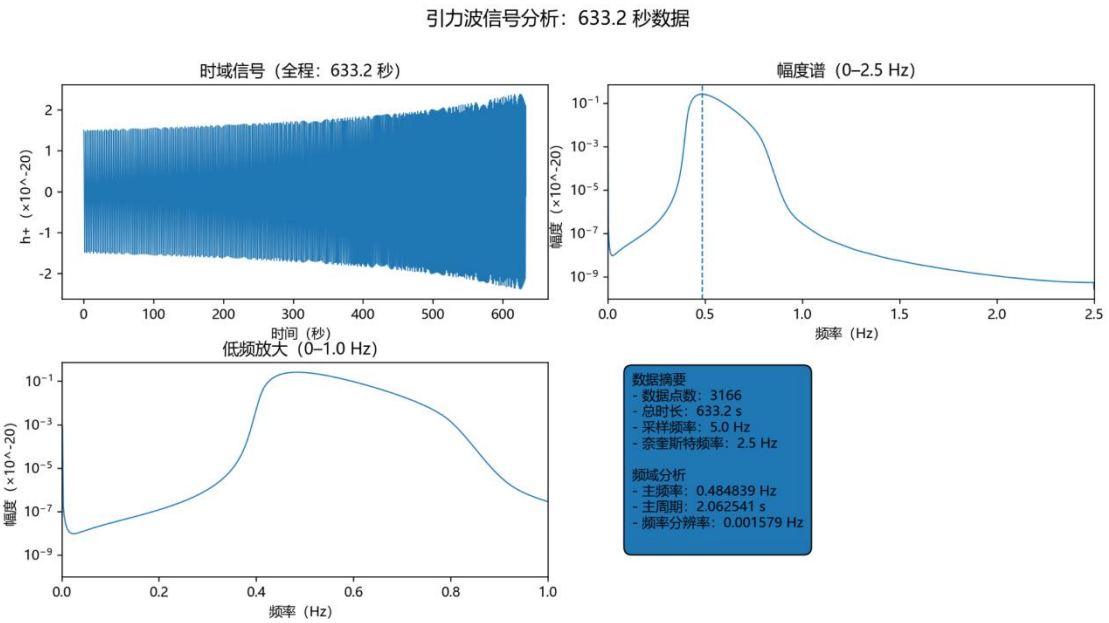
图一

图一包含四个子图：时域信号、全频幅度谱、低频放大、参数摘要。

时域 h+(t)：可见整体振幅随时间并非严格恒定，提示系统处于演化过程。若信号为稳定正弦，时域波形应表现为周期与幅度都基本恒定。

由图一可知，0.8 Hz 附近出现抬升，可能的原因有：
（1）信号幅度在整段 633 s 内明显增长，直接对全段做 FFT 很容易把能量均分，在某些频段显得抬升。
（2）高次谐波（2f、3f…）：如果波形不是完美正弦，例如后期更尖、更不对称，就会在 2 倍频附近出现能量；0.8 Hz 约等于 2×0.4Hz，很值得怀疑是第二个结构。
（3）真实物理阶段切换：如质量转移、潮汐效应、并合前快速演化导致频率内

容突然变宽或出现额外分量。
验证：
1. 在 FFT 前加 Hann 窗



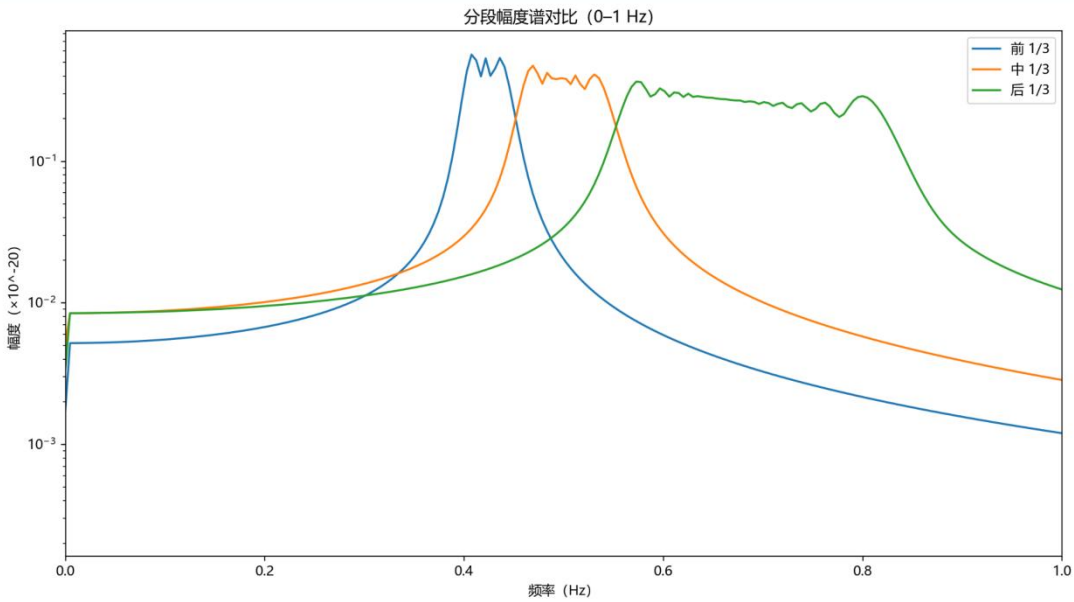
图二

（a）全频幅度谱（0–2.5 Hz）：使用 Hann 窗降低谱泄漏后，谱线上能量主要集中在低频区。按全段 PSD 最大值定义的主峰频率约为 0.484839 Hz，对应周期为 2.062541 s。

（b）低频放大（0–1 Hz）：在接近 0.8 Hz 的区域可观察到能量延伸/抬升趋势。但此处尚不能判断其是否为“独立稳定峰”，因为整段 FFT 会把不同时间段的瞬时频率混合。

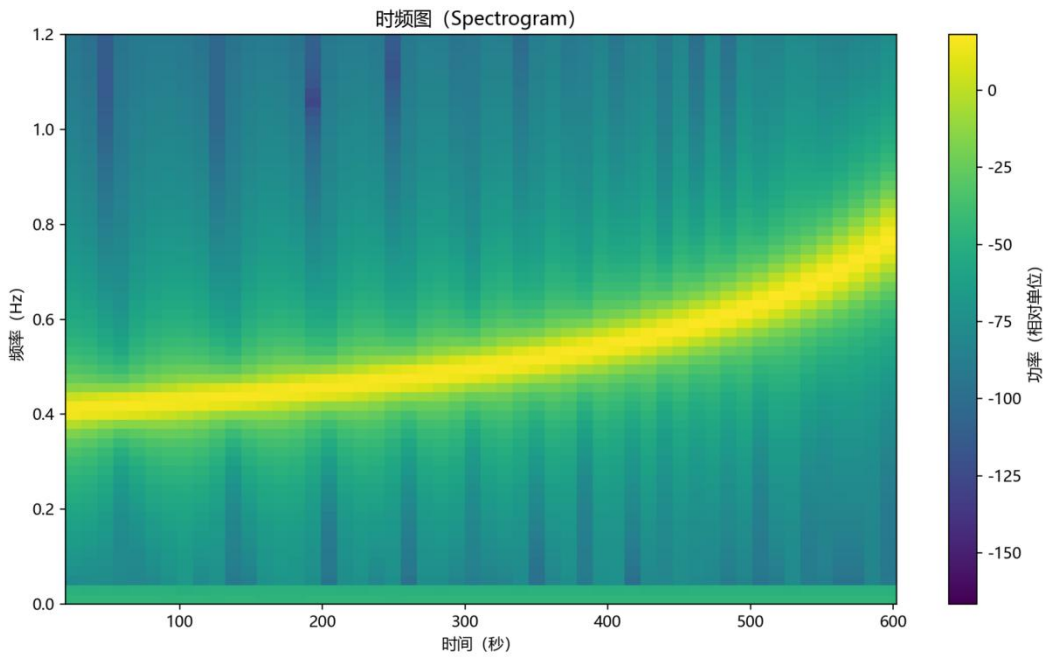
2. 分段 FFT

将全程数据按时间顺序分为三段，前、中、后各约 1/3，分别计算单边幅度谱。如果信号是固定频率，三段主峰应基本一致；如果是扫频，主峰应随时间向高频移动。
本次结果：
前 1/3 主峰 0.407583 Hz，中 1/3 主峰 0.469194 Hz，后 1/3 主峰 0.572917 Hz。主峰频率单调上升，说明信号的主频随时间增加，为典型扫频特征。
这一步是判断 0.8 Hz 结构的重要前提：既然主频在上升，那么后期接近 0.8 Hz 的能量很可能来自“扫频到达该频率范围”。
说明信号不是“固定频率的正弦波”，而是频率随时间变大的信号。0.8 Hz 更像“后段扫频贡献的延伸”，不是一个稳定的独立峰。



图三

下面来验证 0.8 Hz 是否真实存在



图四

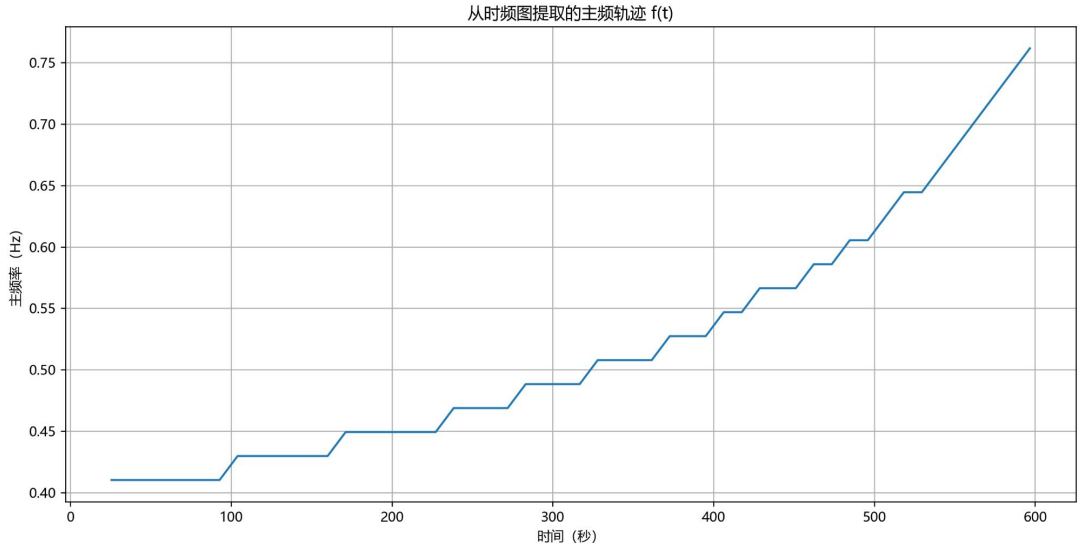
时频图把信号能量在“时间—频率”二维空间展开：横轴时间、纵轴频率、颜色表示功率强度。

若存在独立稳定窄带峰，例如一直在 0.8 Hz 振荡，应在 0.8 Hz 处出现贯穿全程的水平亮线；若为 chirp，则应出现从低频向高频上升的斜亮带。

本次结果显示：最强能量是一条由 0.4 Hz 随时间上升到 0.8 Hz 附近的斜亮带，即 chirp 脊线。因此：0.8 Hz 处的能量主要出现在后期，是主频扫频上升到该频率范围的体现，而不是从头到尾独立存在的稳定窄带峰。

因此，在整段 FFT 里看到的 0.8 Hz 附近抬升，是由于信号非平稳（扫频）、整段 FFT 把不同时间的频率混到一起、谱泄漏一同形成的“尾巴”，不是一个独立的真实窄带新峰。

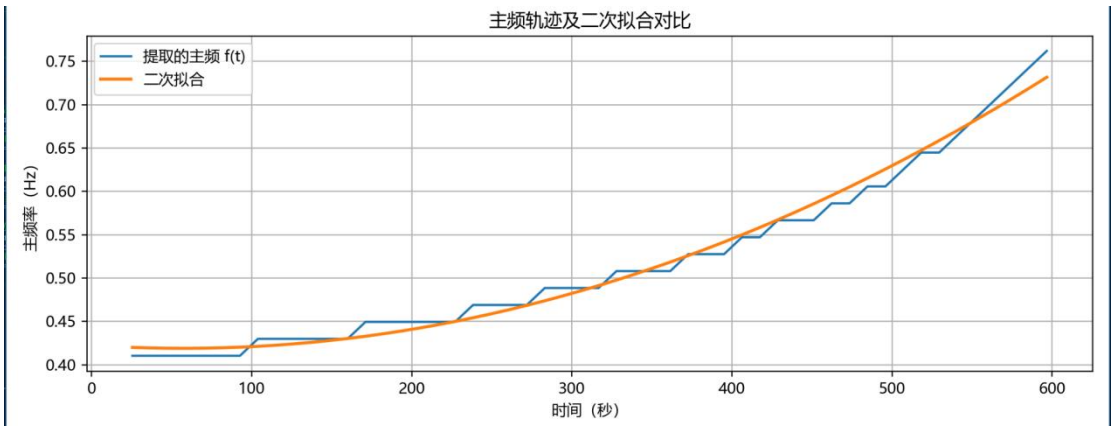
主频轨迹 f(t) 提取:



图五

3.下面进行拟合尝试：

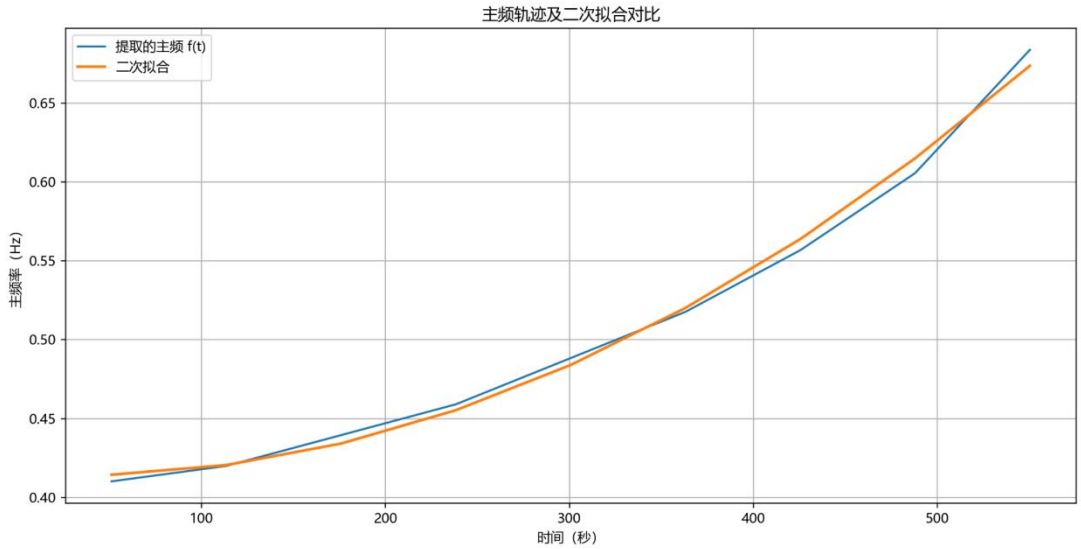
1.二次多项式拟合



图六

二次拟合结果：f(t) = a t^2 + b t + c（NFFT=256）

a = 1.073934e-06, b = -1.229106e-04, c = 4.222291e-01, R^2 = 0.986960



图七

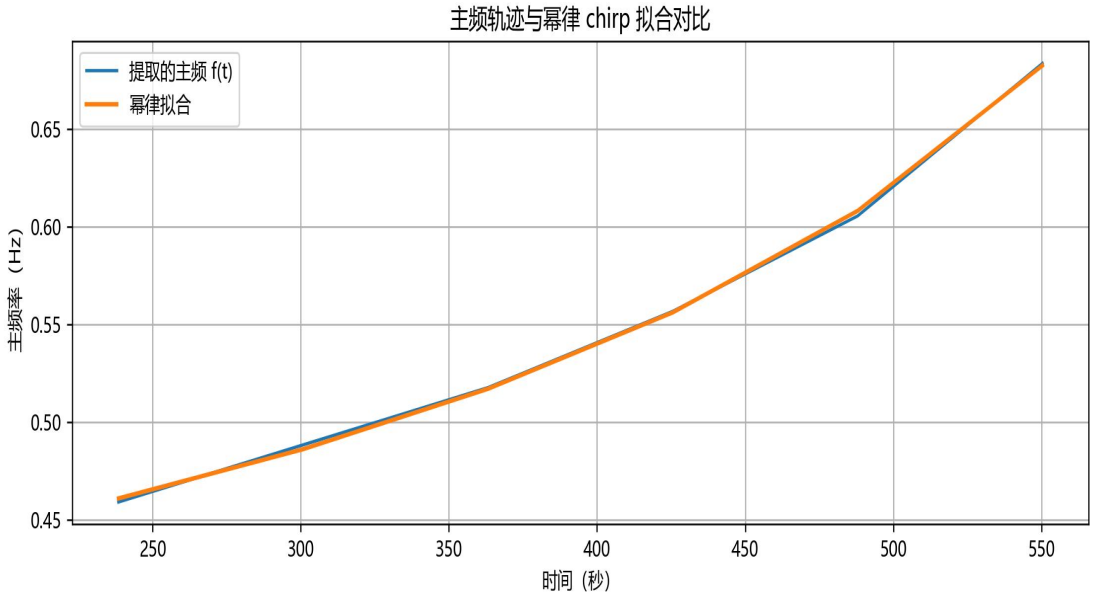
二次拟合结果：f(t) = a t^2 + b t + c （NFFT=512）

a = 9.635788e-07, b = -6.062936e-05, c = 4.149367e-01, R^2 = 0.995192

2.幂律 chirp 拟合

$f(t) = A(t_c - t)^{-p} \quad (t < t_c)$

$\ln f = \ln A - p \ln(t_c - t)$

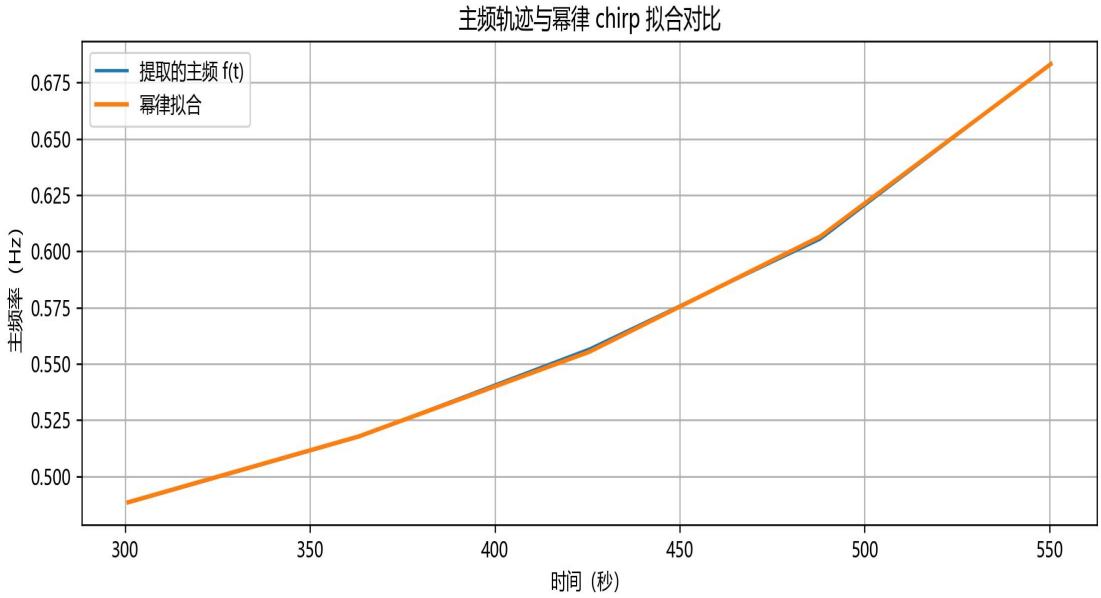


图八

拟合使用的时间范围：238.4 到 550.4 秒

幂律拟合最优结果（log 空间 R^2 最大）：

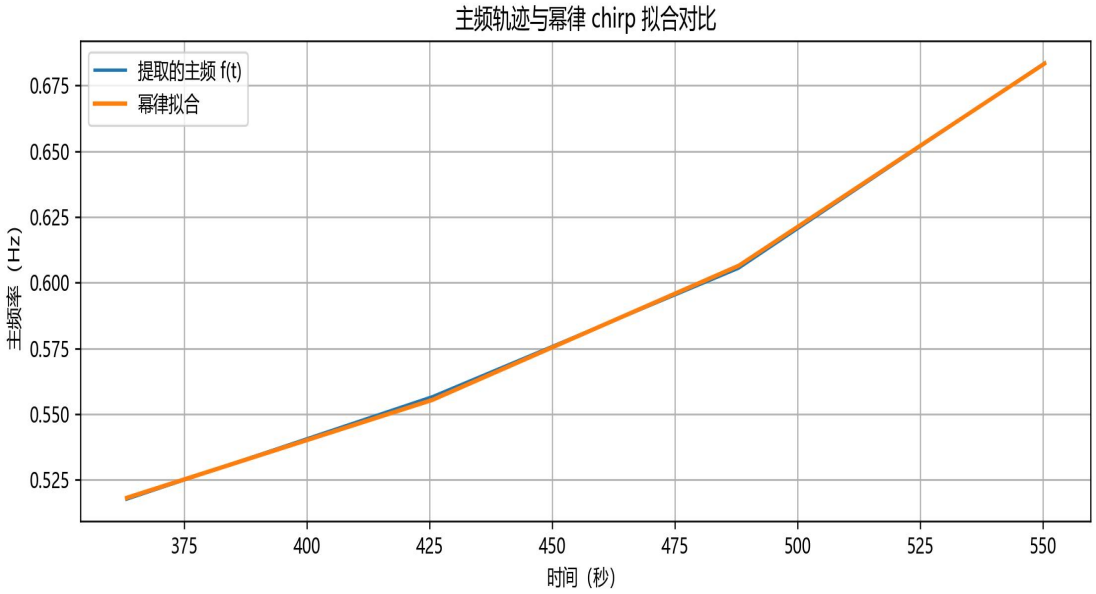
tc = 735.670135 s　A = 5.447758e+00　p = 0.397790　R^2(log) = 0.999440



图九

拟合使用的时间范围：300.8 到 550.4 秒

幂律拟合最优结果（log 空间 R^2 最大）：
tc = 701.250425 s　A = 3.833009e+00　p = 0.343777　R^2(log) = 0.999870



图十

拟合使用的时间范围：363.2 到 550.4 秒

幂律拟合最优结果（log 空间 R^2 最大）：
tc = 696.050325 s　A = 3.625663e+00　p = 0.335012　R^2(log) = 0.999802

最终结论

（1）信号为 chirp（扫频）信号。证据：分段 FFT 主峰频率随时间上升；时频图呈斜脊线；主频轨迹 f(t)单调上升并加速。
（2）0.8 Hz 附近的“出现”在物理上是“后期主频扫频到达该频率范围”的真实体现；但它不是贯穿全程的独立窄带峰。时频图未显示 0.8 Hz 的水平亮线，而是斜脊线在后期经过该频率。
（3）整段 FFT 中 0.8 Hz 附近的“抬升/肩部”主要来自非平稳扫频信号的时变频率混合，并叠加有限时长与窗函数造成的谱泄漏，更像扫频尾巴而非独立峰。
（4）若需要定量描述演化，应优先拟合主频轨迹 f(t)，而不是对整段 PSD 用单一幂律硬拟合。经验层面可用二次多项式快速描述趋势（R^2≈0.993179）；更物理层面可用幂律 chirp f(t)=A(tc-t)^(-p)，并报告拟合区间稳健性（本次 R^2(log)≈0.999 以上）。

对应代码：
1.

```
import re
import numpy as np
import matplotlib.pyplot as plt
import matplotlib as mpl

mpl.rcParams['font.sans-serif'] = ['Microsoft YaHei', 'SimHei']  # 优先微软雅黑，其次黑体
mpl.rcParams['axes.unicode_minus'] = False

infile = r"C:\Users\30126\Desktop\吴鑫星兰州大学\项目\中子星\fort66.txt" dt = 0.2 # 秒
fs = 1.0 / dt                # 采样频率 Hz

rows = []
with open(infile, "r", encoding="utf-8", errors="ignore") as f:
    for line in f:
        # 提取本行的所有数字（支持科学计数法）
        nums = re.findall(r'[-+]?\d*\.?\d+(?:[eE][-+]?\d+)?', line)
        if len(nums) >= 2:
            rows.append((float(nums[0]), float(nums[1])))

data = np.array(rows, dtype=float)
if data.shape[0] == 0:
    raise ValueError("没有读到任何两列数值数据，请检查输入文件格式。")
```

```
h = data[:, 1]  # 第二列：h+（单位：1e-20）
N = len(h)

t = (np.arange(1, N + 1) * dt)

# 3) 预处理：去均值
h0 = h - np.mean(h)

# 4) FFT：频率轴 + 幅度谱（单边）
X = np.fft.rfft(h0)
freq = np.fft.rfftfreq(N, d=dt)

# 幅度谱：让“落在频点上的纯正弦”幅度大致等于时域幅度
amp = np.abs(X) / N
amp_one = amp.copy()
if N % 2 == 0:
    amp_one[1:-1] *= 2.0    # DC 和 Nyquist 不乘 2
else:
    amp_one[1:] *= 2.0

# 5) PSD（periodogram，单边）
#    单位： (h 单位)^2 / Hz
Pxx = (1.0 / (fs * N)) * (np.abs(X) ** 2)
Pxx_one = Pxx.copy()
if N % 2 == 0:
    Pxx_one[1:-1] *= 2.0
else:
    Pxx_one[1:] *= 2.0

# 主峰（排除 DC）
peak_idx = np.argmax(Pxx_one[1:]) + 1
main_f = freq[peak_idx]
main_T = 1.0 / main_f

print(f"数据点数 N = {N}")
print(f"总时长 ={N*dt:.1f} s, 采样频率 fs ={fs:.1f} Hz, 奈奎斯特频率 ={fs/2:.1f}
Hz")
print(f"主频率 ≈ {main_f:.6f} Hz, 对应周期 ≈ {main_T:.6f} s")
print(f"频率分辨率 = {fs/N:.6f} Hz")

# 6) 画图
fig = plt.figure(figsize=(14, 8), dpi=150)
fig.suptitle(f"引力波信号分析：{N*dt:.1f} 秒数据", fontsize=14)
```

```
ax1 = plt.subplot2grid((2, 2), (0, 0))
ax2 = plt.subplot2grid((2, 2), (0, 1))
ax3 = plt.subplot2grid((2, 2), (1, 0))
ax4 = plt.subplot2grid((2, 2), (1, 1))

# 时域
ax1.plot(t, h, linewidth=0.7)
ax1.set_title(f"时域信号（全程：{N*dt:.1f} 秒）")
ax1.set_xlabel("时间（秒）")
ax1.set_ylabel("h+（×10^-20）")

# 幅度谱 0-2.5 Hz
ax2.semilogy(freq, amp_one, linewidth=0.9)
ax2.axvline(main_f, linestyle="--", linewidth=1)
ax2.set_xlim(0, fs / 2)
ax2.set_title(f"幅度谱（0–{fs/2:.1f} Hz）")
ax2.set_xlabel("频率（Hz）")
ax2.set_ylabel("幅度（×10^-20）")

# 低频 0-1.0 Hz（便于看 0.8 Hz）
ax3.semilogy(freq, amp_one, linewidth=0.9)
ax3.set_xlim(0, 1.0)
ax3.set_title("低频放大（0–1.0 Hz）")
ax3.set_xlabel("频率（Hz）")
ax3.set_ylabel("幅度（×10^-20）")

# 摘要框（注意：不要用 monospace，否则会回到不支持中文的字体）
ax4.axis("off")
summary = (
    "数据摘要\n"
    f"- 数据点数：{N}\n"
    f"- 总时长：{N*dt:.1f} s\n"
    f"- 采样频率：{fs:.1f} Hz\n"
    f"- 奈奎斯特频率：{fs/2:.1f} Hz\n\n"
    "频域分析\n"
    f"- 主频率：{main_f:.6f} Hz\n"
    f"- 主周期：{main_T:.6f} s\n"
    f"- 频率分辨率：{fs/N:.6f} Hz\n"
)
ax4.text(0.05, 0.95, summary, va="top", ha="left",
         bbox=dict(boxstyle="round", pad=0.6), fontsize=10)

plt.tight_layout(rect=[0, 0, 1, 0.95])
plt.show()
```

```
# 7) 可选：保存 CSV（中文表头）
np.savetxt("时域数据_dt0p2s.csv",
           np.column_stack([t, h]),
           delimiter=",",
           header="时间_秒,h_plus_(×1e-20)",
           comments="")

np.savetxt("频域谱_PSD.csv",
           np.column_stack([freq, Pxx_one, amp_one]),
           delimiter=",",
           header="频率_Hz,功率谱密度_((1e-20)^2/Hz),幅度谱_(×1e-20)",
           comments="")

print("已保存：时域数据_dt0p2s.csv 与 频域谱_PSD.csv")
```

2.
将上述代码的

```
X = np.fft.rfft(h0)
freq = np.fft.rfftfreq(N, d=dt)
```

换成

```
# 加 Hann 窗，降低谱泄漏
w = np.hanning(N)              # Hann 窗
h_win = h0 * w

X = np.fft.rfft(h_win)
freq = np.fft.rfftfreq(N, d=dt)

# 窗函数幅度修正：补偿 Hann 窗把幅度压小的问题
coherent_gain = np.mean(w)     # Hann 窗的相干增益
X = X / coherent_gain
```

即可

3.

```
import re
import numpy as np
import matplotlib.pyplot as plt
import matplotlib as mpl

# 0) Matplotlib 中文字体设置（解决中文乱码/缺字警告）
mpl.rcParams['font.sans-serif'] = ['Microsoft YaHei', 'SimHei']
mpl.rcParams['axes.unicode_minus'] = False

# 1) 读取两列数据（忽略原时间列，用第二列 h+）
```

```
infile = r"C:\Users\30126\Desktop\吴鑫星兰州大学\项目\中子星\fort66.txt"
dt = 0.2
fs = 1.0 / dt

rows = []
with open(infile, "r", encoding="utf-8", errors="ignore") as f:
    for line in f:
        nums = re.findall(r'[-+]?\d*\.?\d+(?:[eE][-+]?\d+)?', line)
        if len(nums) >= 2:
            rows.append((float(nums[0]), float(nums[1])))

data = np.array(rows, dtype=float)
if data.shape[0] == 0:
    raise ValueError("没有读到任何两列数值数据，请检查输入文件格式。")

h = data[:, 1]          # 第二列：h+（单位：1e-20）
N = len(h)
t = np.arange(1, N + 1) * dt  # 替换时间列：0.2, 0.4, ..., N*0.2

# 2) 预处理：去均值
h0 = h - np.mean(h)

# 3) 加 Hann 窗（降低谱泄漏）+ 幅度修正
w = np.hanning(N)
h_win = h0 * w

X = np.fft.rfft(h_win)
freq = np.fft.rfftfreq(N, d=dt)

# Hann 窗相干增益修正：补偿幅度被窗压小
coherent_gain = np.mean(w)
X = X / coherent_gain

# 4) 单边幅度谱
amp = np.abs(X) / N
amp_one = amp.copy()
if N % 2 == 0:
    amp_one[1:-1] *= 2.0
else:
    amp_one[1:] *= 2.0

# 5) 单边 PSD（periodogram）
Pxx = (1.0 / (fs * N)) * (np.abs(X) ** 2)
Pxx_one = Pxx.copy()
```

```
if N % 2 == 0:
    Pxx_one[1:-1] *= 2.0
else:
    Pxx_one[1:] *= 2.0

# 主峰（排除 DC）
peak_idx = np.argmax(Pxx_one[1:]) + 1
main_f = freq[peak_idx]
main_T = 1.0 / main_f

print(f"数据点数 N = {N}")
print(f"总时长 = {N*dt:.1f} s, 采样频率 fs = {fs:.1f} Hz, 奈奎斯特频率 = {fs/2:.1f} Hz")
print(f"主频率 ≈ {main_f:.6f} Hz, 对应周期 ≈ {main_T:.6f} s")
print(f"频率分辨率 = {fs/N:.6f} Hz")

# 6) 画图：时域 + 幅度谱 + 低频放大 + 摘要
fig = plt.figure(figsize=(14, 8), dpi=150)
fig.suptitle(f"引力波信号分析：{N*dt:.1f} 秒数据", fontsize=14)

ax1 = plt.subplot2grid((2, 2), (0, 0))
ax2 = plt.subplot2grid((2, 2), (0, 1))
ax3 = plt.subplot2grid((2, 2), (1, 0))
ax4 = plt.subplot2grid((2, 2), (1, 1))

# 时域
ax1.plot(t, h, linewidth=0.7)
ax1.set_title(f"时域信号（全程：{N*dt:.1f} 秒）")
ax1.set_xlabel("时间（秒）")
ax1.set_ylabel("h+（×10^-20）")

# 幅度谱 0-2.5 Hz
ax2.semilogy(freq, amp_one, linewidth=0.9)
ax2.axvline(main_f, linestyle="--", linewidth=1)
ax2.set_xlim(0, fs / 2)
ax2.set_title(f"幅度谱（0 - {fs/2:.1f} Hz）")
ax2.set_xlabel("频率（Hz）")
ax2.set_ylabel("幅度（×10^-20）")

# 低频 0-1.0 Hz
ax3.semilogy(freq, amp_one, linewidth=0.9)
ax3.set_xlim(0, 1.0)
ax3.set_title("低频放大（0 - 1.0 Hz）")
ax3.set_xlabel("频率（Hz）")
```

```
ax3.set_ylabel("幅度（×10^-20）")

# 摘要框（不要用 monospace）
ax4.axis("off")
summary = (
    "数据摘要\n"
    f"- 数据点数：{N}\n"
    f"- 总时长：{N*dt:.1f} s\n"
    f"- 采样频率：{fs:.1f} Hz\n"
    f"- 奈奎斯特频率：{fs/2:.1f} Hz\n\n"
    "频域分析\n"
    f"- 主频率：{main_f:.6f} Hz\n"
    f"- 主周期：{main_T:.6f} s\n"
    f"- 频率分辨率：{fs/N:.6f} Hz\n"
)
ax4.text(0.05, 0.95, summary, va="top", ha="left",
         bbox=dict(boxstyle="round", pad=0.6), fontsize=10)

plt.tight_layout(rect=[0, 0, 1, 0.95])
plt.show()

# 7) 分三段 FFT：验证是否 chirp（频率随时间上升）
def one_sided_amp(x, dt):
    Nn = len(x)
    Xn = np.fft.rfft(x)
    fn = np.fft.rfftfreq(Nn, d=dt)
    an = np.abs(Xn) / Nn
    if Nn % 2 == 0:
        an[1:-1] *= 2.0
    else:
        an[1:] *= 2.0
    return fn, an

seg = N // 3
x1 = h0[:seg]
x2 = h0[seg:2*seg]
x3 = h0[2*seg:]

f1, a1 = one_sided_amp(x1, dt)
f2, a2 = one_sided_amp(x2, dt)
f3, a3 = one_sided_amp(x3, dt)

plt.figure(figsize=(10, 6), dpi=150)
plt.semilogy(f1, a1, label="前 1/3")
```

```
plt.semilogy(f2, a2, label="中 1/3")
plt.semilogy(f3, a3, label="后 1/3")
plt.xlim(0, 1.0)
plt.xlabel("频率（Hz）")
plt.ylabel("幅度（×10^-20）")
plt.title("分段幅度谱对比（0－1 Hz）")
plt.legend()
plt.tight_layout()
plt.show()

def peak_freq(fn, an):
    idx = np.argmax(an[1:]) + 1
    return fn[idx], an[idx]

p1 = peak_freq(f1, a1)
p2 = peak_freq(f2, a2)
p3 = peak_freq(f3, a3)

print("前三段主峰（频率 Hz, 幅度）：")
print("前 1/3:", p1)
print("中 1/3:", p2)
print("后 1/3:", p3)

# 8) 可选：保存 CSV
np.savetxt("时域数据_dt0p2s.csv",
           np.column_stack([t, h]),
           delimiter=",",
           header="时间_秒,h_plus_(×1e-20)",
           comments="")

np.savetxt("频域谱_PSD.csv",
           np.column_stack([freq, Pxx_one, amp_one]),
           delimiter=",",
           header="频率_Hz,功率谱密度_((1e-20)^2/Hz),幅度谱_(×1e-20)",
           comments="")

print("已保存：时域数据_dt0p2s.csv 与 频域谱_PSD.csv")
```

4.

在上述代码最后加以下代码即可

```
# 时频图：判断 0.8 Hz 是“水平线”还是“扫频脊线”
plt.figure(figsize=(10,6), dpi=150)
```

```
# NFFT：每段 FFT 长度（越大频率分辨率越好，但时间分辨率变差）
NFFT = 256
noverlap = 200

Pxx, freqs, bins, im = plt.specgram(h0, NFFT=NFFT, Fs=fs, noverlap=noverlap)
plt.ylim(0, 1.2)
plt.xlabel("时间（秒）")
plt.ylabel("频率（Hz）")
plt.title("时频图（Spectrogram）")
plt.colorbar(label="功率（相对单位）")
plt.tight_layout()
plt.show()
```

5.

```
import re
import numpy as np
import matplotlib.pyplot as plt
import matplotlib as mpl

# 0) Matplotlib 中文字体设置（解决中文乱码/缺字警告）
mpl.rcParams['font.sans-serif'] = ['Microsoft YaHei', 'SimHei']
mpl.rcParams['axes.unicode_minus'] = False

# 1) 读取两列数据（忽略原时间列，用第二列 h+）
infile = r"C:\Users\30126\Desktop\吴鑫星兰州大学\项目\中子星\fort66.txt"

dt = 0.2
fs = 1.0 / dt

rows = []
with open(infile, "r", encoding="utf-8", errors="ignore") as f:
    for line in f:
        nums = re.findall(r'[-+]?\d*\.?\d+(?:[eE][-+]?\d+)?', line)
        if len(nums) >= 2:
            rows.append((float(nums[0]), float(nums[1])))

data = np.array(rows, dtype=float)
if data.shape[0] == 0:
    raise ValueError("没有读到任何两列数值数据，请检查输入文件格式。")

h = data[:, 1]                    # 第二列：h+（单位：1e-20）
N = len(h)
t = np.arange(1, N + 1) * dt      # 替换时间列：0.2, 0.4, ..., N*0.2
```

```
# 2) 预处理：去均值
h0 = h - np.mean(h)

# 3) Hann 窗 + FFT（降低谱泄漏）
w = np.hanning(N)
h_win = h0 * w

X = np.fft.rfft(h_win)
freq = np.fft.rfftfreq(N, d=dt)

# Hann 窗相干增益修正：补偿幅度被窗压小
coherent_gain = np.mean(w)
X = X / coherent_gain

# ---- 单边幅度谱 ----
amp = np.abs(X) / N
amp_one = amp.copy()
if N % 2 == 0:
    amp_one[1:-1] *= 2.0
else:
    amp_one[1:] *= 2.0

Pxx = (1.0 / (fs * N)) * (np.abs(X) ** 2)
Pxx_one = Pxx.copy()
if N % 2 == 0:
    Pxx_one[1:-1] *= 2.0
else:
    Pxx_one[1:] *= 2.0

# 主峰（排除 DC）
peak_idx = np.argmax(Pxx_one[1:]) + 1
main_f = freq[peak_idx]
main_T = 1.0 / main_f

print(f"数据点数 N = {N}")
print(f"总时长 ={N*dt:.1f} s, 采样频率 fs ={fs:.1f} Hz, 奈奎斯特频率 ={fs/2:.1f}
Hz")
print(f"主频率 ≈ {main_f:.6f} Hz, 对应周期 ≈ {main_T:.6f} s")
print(f"频率分辨率 = {fs/N:.6f} Hz")

# 4) 总览图：时域 + 幅度谱 + 低频放大 + 摘要
fig = plt.figure(figsize=(14, 8), dpi=150)
fig.suptitle(f"引力波信号分析：{N*dt:.1f} 秒数据", fontsize=14)
ax1 = plt.subplot2grid((2, 2), (0, 0))
```

```
ax2 = plt.subplot2grid((2, 2), (0, 1))
ax3 = plt.subplot2grid((2, 2), (1, 0))
ax4 = plt.subplot2grid((2, 2), (1, 1))

# 时域
ax1.plot(t, h, linewidth=0.7)
ax1.set_title(f"时域信号（全程：{N*dt:.1f} 秒）")
ax1.set_xlabel("时间（秒）")
ax1.set_ylabel("h+（×10^-20）")

# 幅度谱 0-2.5 Hz
ax2.semilogy(freq, amp_one, linewidth=0.9)
ax2.axvline(main_f, linestyle="--", linewidth=1)
ax2.set_xlim(0, fs / 2)
ax2.set_title(f"幅度谱（0–{fs/2:.1f} Hz）")
ax2.set_xlabel("频率（Hz）")
ax2.set_ylabel("幅度（×10^-20）")

# 低频 0-1.0 Hz
ax3.semilogy(freq, amp_one, linewidth=0.9)
ax3.set_xlim(0, 1.0)
ax3.set_title("低频放大（0–1.0 Hz）")
ax3.set_xlabel("频率（Hz）")
ax3.set_ylabel("幅度（×10^-20）")

# 摘要框
ax4.axis("off")
summary = (
    "数据摘要\n"
    f"- 数据点数：{N}\n"
    f"- 总时长：{N*dt:.1f} s\n"
    f"- 采样频率：{fs:.1f} Hz\n"
    f"- 奈奎斯特频率：{fs/2:.1f} Hz\n\n"
    "频域分析\n"
    f"- 主频率：{main_f:.6f} Hz\n"
    f"- 主周期：{main_T:.6f} s\n"
    f"- 频率分辨率：{fs/N:.6f} Hz\n"
)
ax4.text(0.05, 0.95, summary, va="top", ha="left",
         bbox=dict(boxstyle="round", pad=0.6), fontsize=10)

plt.tight_layout(rect=[0, 0, 1, 0.95])
plt.show()  # 关闭这个窗口后才会继续往下运行
# 5) 分三段 FFT：验证是否 chirp（频率随时间上升）
```

```
def one_sided_amp(x, dt):
    Nn = len(x)
    Xn = np.fft.rfft(x)
    fn = np.fft.rfftfreq(Nn, d=dt)
    an = np.abs(Xn) / Nn
    if Nn % 2 == 0:
        an[1:-1] *= 2.0
    else:
        an[1:] *= 2.0
    return fn, an

seg = N // 3
x1 = h0[:seg]
x2 = h0[seg:2*seg]
x3 = h0[2*seg:]

f1, a1 = one_sided_amp(x1, dt)
f2, a2 = one_sided_amp(x2, dt)
f3, a3 = one_sided_amp(x3, dt)

plt.figure(figsize=(10, 6), dpi=150)
plt.semilogy(f1, a1, label="前 1/3")
plt.semilogy(f2, a2, label="中 1/3")
plt.semilogy(f3, a3, label="后 1/3")
plt.xlim(0, 1.0)
plt.xlabel("频率（Hz）")
plt.ylabel("幅度（×10^-20）")
plt.title("分段幅度谱对比（0–1 Hz）")
plt.legend()
plt.tight_layout()
plt.show()

def peak_freq(fn, an):
    idx = np.argmax(an[1:]) + 1
    return fn[idx], an[idx]

p1 = peak_freq(f1, a1)
p2 = peak_freq(f2, a2)
p3 = peak_freq(f3, a3)

print("前三段主峰（频率 Hz, 幅度）：")
print("前 1/3:", p1)
print("中 1/3:", p2)
print("后 1/3:", p3)
```

```
# 6) 时频图 Spectrogram + 提取主频脊线 f(t)
plt.figure(figsize=(10, 6), dpi=150)

NFFT = 256
noverlap = 200

Pxx_spec, freqs_spec, bins_spec, im = plt.specgram(
    h0, NFFT=NFFT, Fs=fs, noverlap=noverlap
)

plt.ylim(0, 1.2)
plt.xlabel("时间（秒）")
plt.ylabel("频率（Hz）")
plt.title("时频图（Spectrogram）")
plt.colorbar(label="功率（相对单位）")
plt.tight_layout()
plt.show()

# --- 提取主频脊线：每个时间窗取最大功率对应的频率 ---
ridge_idx = np.argmax(Pxx_spec, axis=0)     # axis=0：对频率方向取最大
f_ridge = freqs_spec[ridge_idx]

plt.figure(figsize=(10, 4), dpi=150)
plt.plot(bins_spec, f_ridge)
plt.xlabel("时间（秒）")
plt.ylabel("主频率（Hz）")
plt.title("从时频图提取的主频轨迹 f(t)")
plt.grid(True)
plt.tight_layout()
plt.show()

print("主频率范围：", float(np.min(f_ridge)), "到", float(np.max(f_ridge)), "Hz")

# 7) 保存 CSV
np.savetxt("时域数据_dt0p2s.csv",
           np.column_stack([t, h]),
           delimiter=",",
           header="时间_秒,h_plus_(×1e-20)",
           comments="")

np.savetxt("频域谱_PSD.csv",
           np.column_stack([freq, Pxx_one, amp_one]),
           delimiter=",",
           header="频率_Hz,功率谱密度_((1e-20)^2/Hz),幅度谱_(×1e-20)",
```

```
            comments="")

np.savetxt("主频轨迹_f_of_t.csv",
            np.column_stack([bins_spec, f_ridge]),
            delimiter=",",
            header="时间_秒,主频率_Hz",
            comments="")

print("已保存：时域数据_dt0p2s.csv、频域谱_PSD.csv、主频轨迹_f_of_t.csv")
```

6.

在上述代码最后加以下代码即可：

```
# 对主频轨迹 f(t) 做二次拟合：f(t)=a t^2 + b t + c
# bins_spec: 时间点（秒），f_ridge: 对应主频（Hz）

t_fit = bins_spec
f_fit = f_ridge

# 二次拟合（返回 [a,b,c]）
coef = np.polyfit(t_fit, f_fit, 2)
a, b, c = coef
f_pred = np.polyval(coef, t_fit)

# 拟合优度 R^2
ss_res = np.sum((f_fit - f_pred)**2)
ss_tot = np.sum((f_fit - np.mean(f_fit))**2)
r2 = 1 - ss_res/ss_tot

print("\n二次拟合结果：f(t) = a t^2 + b t + c")
print(f"a = {a:.6e}, b = {b:.6e}, c = {c:.6e},  R^2 = {r2:.6f}")

plt.figure(figsize=(10,4), dpi=150)
plt.plot(t_fit, f_fit, label="提取的主频 f(t)")
plt.plot(t_fit, f_pred, label="二次拟合", linewidth=2)
plt.xlabel("时间（秒）")
plt.ylabel("主频率（Hz）")
plt.title("主频轨迹及二次拟合对比")
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()
```

7.
将时频图代码中的 NFFT = 256 改为 NFFT = 512

8. 9. 10.

```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib as mpl

# 中文字体（可选）
mpl.rcParams['font.sans-serif'] = ['Microsoft YaHei', 'SimHei']
mpl.rcParams['axes.unicode_minus'] = False

# 确保这个文件在当前目录，或者写绝对路径
csv_path = r"主频轨迹_f_of_t.csv"   ）

data = np.loadtxt(csv_path, delimiter=",", skiprows=1)
t_data = data[:, 0]
f_data = data[:, 1]

# 只取后段数据来拟合（改这个阈值，比如 250、300、350）=====
t_start_fit = 200.0
mask = t_data >= t_start_fit
t_fit = t_data[mask]
f_fit = f_data[mask]

mask2 = f_fit > 0
t_fit = t_fit[mask2]
f_fit = f_fit[mask2]

t_end = float(np.max(t_fit))
print("拟合使用的时间范围：", float(np.min(t_fit)), "到", t_end, "秒")

tc_min = t_end + 5.0
tc_max = t_end + 500.0
num_tc = 2000

tc_grid = np.linspace(tc_min, tc_max, num_tc)

best = None  # (R2, tc, A, p)
eps = 1e-12

for tc in tc_grid:
    tau = tc - t_fit
    if np.any(tau <= 0):
```

```
        continue

    x = np.log(tau + eps)
    y = np.log(f_fit + eps)

    m, b = np.polyfit(x, y, 1)   # y = b + m x
    y_pred = m * x + b

    ss_res = np.sum((y - y_pred) ** 2)
    ss_tot = np.sum((y - np.mean(y)) ** 2)
    R2 = 1 - ss_res / ss_tot

    p = -m
    A = np.exp(b)

    if p <= 0:
        continue

    if (best is None) or (R2 > best[0]):
        best = (R2, tc, A, p)

if best is None:
    raise RuntimeError("没有找到合适的幂律拟合。尝试把 t_start_fit 调大，或扩大 tc 搜索范围。")

R2_best, tc_best, A_best, p_best = best
print("\n幂律拟合最优结果（log 空间 R^2 最大）：")
print(f"tc = {tc_best:.6f} s")
print(f"A  = {A_best:.6e}")
print(f"p  = {p_best:.6f}")
print(f"R^2(log) = {R2_best:.6f}")

f_model = A_best * (tc_best - t_fit) ** (-p_best)

plt.figure(figsize=(10,4), dpi=150)
plt.plot(t_fit, f_fit, label="提取的主频 f(t)")
plt.plot(t_fit, f_model, label="幂律拟合", linewidth=2)
plt.xlabel("时间（秒）")
plt.ylabel("主频率（Hz）")
plt.title("主频轨迹与幂律 chirp 拟合对比")
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()
```

```
tau_best = tc_best - t_fit
x_best = np.log(tau_best + eps)
y_best = np.log(f_fit + eps)
y_line = np.log(A_best + eps) - p_best * x_best

plt.figure(figsize=(10,4), dpi=150)
plt.scatter(x_best, y_best, s=12, label="数据：log f")
plt.plot(x_best, y_line, label="线性拟合：logA - p log(tau)", linewidth=2)
plt.xlabel("log(tc - t)")
plt.ylabel("log(f)")
plt.title("log-log 检验图（应接近直线）")
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()
```